LEAN SOFTWARE DEVELOPMENT

Team Members:

Prasiddha Raj Poudel (THA080BCT027)
Pratik Pant (THA080BCT028)
Purnima Bhandari (THA080BCT029)

Department of Electronics and Computer Engineering
Institute of Engineering, Thapathali Campus

July 9, 2026

Presentation Outline

- Origins and Philosophy of Lean
- Traditional Problems and Solution
- The Seven Process Wastes
- The 7 Core Lean Principles
- Tools and Practical Comparison
- Advantages, Disadvantages, and Case Study

What is Lean Software Development?

Lean Software Development is a software development approach that focuses on delivering maximum value to customers while minimizing waste.

Key Points

- Inspired by Lean Manufacturing and the Toyota Production System
- Adapted for software development by Mary and Tom Poppendieck
- Emphasizes efficiency, quality, and customer satisfaction
- Supports Agile and Continuous Delivery practice

Why Lean Was Introduced

Problems with Traditional Development

- Traditional software development required extensive documentation before coding began.
- A large amount of time and effort was spent creating documents.
- Documents often became outdated as project requirements changed.
- This reduced efficiency and slowed down software delivery.
- Feedback was often received late in the development process.
- Late changes increased project cost, effort, and delays.

Lean Solution

- Lean focuses on delivering value to customers as quickly as possible.
- Working software is released in smaller and more frequent increments.
- Continuous feedback helps teams improve and adapt to changes.
- Waste is minimized, improving overall efficiency and productivity.

The Seven Wastes in Software Development

Seven Types of Waste

1. **Partially Done Work** - Incomplete tasks that are not yet useful
2. **Extra Features** - Features that customers do not need
3. **Relearning** - Loss of knowledge requiring repeated effort
4. **Task Switching** - Reduced productivity from multitasking
5. **Waiting** - Delays between development activities
6. **Handoffs** - Information loss during communication
7. **Defects** - Errors that require correction and rework

The 7 principles of lean software development

Eliminate Waste

If it doesn't add value for the user — remove it.

- Remove unused features, duplicate steps, and unnecessary waiting
- Avoid writing code that nobody asked for
- Less clutter means faster delivery and a cleaner codebase
- Waste includes: delays, defects, extra processes, and over-engineering

Build Quality In

Catch problems early — prevention is cheaper than repair.

- Test code as it is written, not after the entire product is done
- Use automated test suites so every change is verified instantly
- Code reviews and pair programming catch issues before they ship
- A bug fixed in development costs 10$\times$ less than one found in production

Create Knowledge

Learn continuously and share what you know.

- Document decisions and the reasons behind them
- Hold short retrospectives after each sprint or milestone
- Maintain a shared wiki so lessons reach the whole team
- Don’t repeat the same mistakes i.e. knowledge compounds over time

Defer Commitment

Delay big decisions until you have enough information.

- Keep options open as long as it's practical to do so
- Avoid expensive, hard-to-reverse choices made on early assumptions
- Gather data and user feedback before locking in an architecture
- Decide at 'the last responsible moment' — not the last possible moment

Deliver Fast

Release early, release often - get feedback sooner.

- Break large projects into short, shippable work cycles
- Deliver small updates rather than one massive release
- Fast delivery surfaces problems before they snowball
- Speed reduces risk i.e. smaller batches are easier to test and roll back

Respect People

Trust your team and give them real ownership.

- Let developers make decisions about their own work
- Listen to feedback and act on what the team surfaces
- Protect work-life balance to sustain long-term performance
- Psychological safety drives creativity and honest communication

Optimize the Whole

Look at the entire pipeline, not just one step.

- Speeding up one stage can create bottlenecks elsewhere
- Map the full journey from idea to production deployment
- Remove handoff delays between teams (dev $\rightarrow$ QA $\rightarrow$ ops)
- Global optimisation beats local improvements every time

Tools & Techniques in Lean

Value Stream Mapping (VSM)

- Diagrams every step from requirement to delivery
- Highlights delays, waste, and unnecessary handoffs

Kanban Board

- Visualizes tasks: To Do, In Progress, Done
- Limits work-in-progress to prevent overload

Pull-Based Scheduling

- Work begins only when team has capacity
- Prevents bottlenecks from piling up downstream

Continuous Integration

- Code tested automatically after every change
- Defects caught immediately, not at release

Lean vs Agile Development

Lean is a philosophy; Agile is a methodology

Both reject heavy upfront planning and documentation

Agile Manifesto (2001) was shaped by Lean thinking

Scrum implements Lean through sprints and roles

- Roles: Product Owner, Scrum Master, Dev Team
- Fixed iterations: typically 1–4 week sprints

Lean has no fixed iterations or prescribed roles

Key shared goal: deliver working software faster

Advantages

- Reduces time from requirement to working software
- Defects caught early cost significantly less to fix
- Removes steps that add no customer value
- Frequent releases allow continuous user feedback
- Developers given autonomy produce higher quality work
- Cleaner code reduces long-term maintenance cost

Disadvantages

- Requires full organizational commitment to work
- Difficult without experienced and disciplined team members
- Reduced documentation risks long-term maintainability
  - System knowledge becomes dependent on individuals
  - New members struggle without proper documentation
- Over-reliance on key members increases project risk
- Progress and improvement difficult to measure objectively

Case Study: Toyota Production System

- Large batch production hid defects until very late
- Value stream mapping revealed primary delay points
  - Handoffs between stations were main bottleneck
- Kanban introduced to pull work by actual capacity
- Defects surfaced and resolved immediately at each stage
- Software teams later applied same principles to code delivery

Conclusion

- Lean targets root causes of software project failure
- Waste elimination improves both speed and quality
- Agile and Lean share the same foundational goals
- Documentation remains critical despite Lean's speed focus
- Team capability determines how well Lean is applied
- Lean thinking applies across any development context

Thank You

Questions & Answers